

Práctica P1

Navegación en entornos complejos

Mapeo y cobertura, navegación punto a punto y teleoperación

Asignatura	Robótica de Servicios
Curso	2026/2027
Autor confirmado	Alejandro Lavandeira
Entorno	Python 3.12.3, Pygame 2.6.1

Nota de entrega: completar en portada los demás integrantes del equipo antes de subir el ZIP.

Resumen

Se presenta una solución integral para una aspiradora con odometría y tres sensores de contacto. El seguidor de paredes inicial se sustituyó por cobertura completa sobre una rejilla del espacio de configuración; la navegación utiliza A* y la teleoperación guarda waypoints que se reproducen con el mismo planificador. La validación recorre los cuatro escenarios proporcionados: alcanza el 100% de las posiciones transitables, completa rutas largas con error final nulo y no registra colisiones durante navegación ni reproducción.

1. Requisitos y diseño global

El enunciado solicita: (A) distinguir celdas ocupadas y libres y cubrir la superficie; (B) desplazarse entre dos puntos arbitrarios sorteando obstáculos; y (C) teleoperar, guardar rutas y recuperarlas. La evaluación usa también entornos no proporcionados, por lo que se evitó codificar trayectorias específicas.

Arquitectura

Capa	Responsabilidad	Implementación
Simulador	Dinámica, contacto y odometría	rds2026machines.py
Modelo	Rejilla segura para el centro del robot	OccupancyGrid
Exploración	Descubrir y cubrir todo lo alcanzable	CompleteCoverageExplorer
Planificación	Ruta óptima conocida	A* + compresión cardinal
Control	Giro y avance con realimentación	MotionController
Persistencia	Mapas y waypoints versionados	JSON + escritura atómica

Correcciones del simulador

Se eliminaron contenedores mutables compartidos entre robots y planos, se inicializaron posición y orientación del sensor por instancia, se corrigió la coordenada Y de recarga y se anuló el ruido trigonométrico en movimientos cardinales. Además, el cálculo de contacto futuro se realiza en una sola conversión a rectángulo: el doble redondeo anterior hacía algunas aristas transitables sólo en un sentido.

La solución sólo usa el oráculo geométrico del plano en la batería de evaluación. La exploración operativa decide con posición odométrica y contactos; no consulta los objetos del entorno.

2. Apartado A: mapeo y cobertura

Representación

El explorador desconoce las dimensiones del piso: conserva poses libres, contactos y celdas cubiertas en conjuntos dispersos de coordenadas odométricas. Terminada la exploración, calcula los mínimos y máximos observados en ambos ejes y sólo entonces crea la matriz densa con su origen. El simulador conoce el tamaño para dibujar, pero no se lo entrega al algoritmo. El mapa distingue poses libres del centro cada 0,5 celdas y suelo cubierto por la huella 2x2; además conserva el estado desconocido separado del obstáculo.

Algoritmo de cobertura

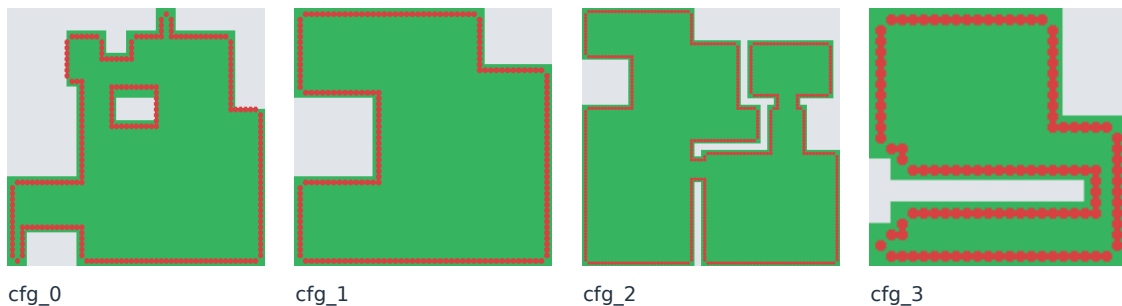
Se emplea exploración en línea mediante un árbol de expansión con retroceso (DFS). Desde cada pose se prueban los cuatro vecinos cardinales a distancia 0,5, que coincide con un paso físico del simulador. El sensor frontal se consulta antes de mandar el avance: una pose libre se añade al árbol y una ocupada se registra sin impacto. Al agotar los vecinos se vuelve al padre. El algoritmo termina cuando la pila queda vacía.

La primera implementación sólo ramificaba en centros enteros. Aunque atravesaba poses intermedias, no podía explorar desde ellas y omitía cuatro celdas de superficie en `cfg_0`. La nueva rejilla de medio paso cubre todas las poses y toda la huella alcanzable.

Propiedades y complejidad

Para V posiciones alcanzables y E transiciones candidatas, el coste de búsqueda es $O(V+E)$ y la memoria $O(V)$. El recorrido no es de longitud mínima, pero sí finito, reproducible y completo sobre la discretización. Frente al seguimiento de pared, puede abandonar contornos, cubrir interiores y cerrar ramas mediante backtracking.

Visualización de mapas resultantes



Verde: superficie barrida por la huella; rojo: pose de contacto observada; gris: desconocido o inaccesible. La capa JSON conserva por separado las poses navegables.

3. Apartado B: navegación punto a punto

Planificación

Los puntos arbitrarios se proyectan a la pose de medio paso más próxima dentro de la componente conexa del origen, y se informa de la distancia de ajuste. A* busca sobre vecinos cardinales con coste unitario y heurística Manhattan, admisible y consistente; por tanto devuelve un camino de coste mínimo en la rejilla medida.

Reducción de giros

Después de A* se eliminan waypoints colineales y se conservan sólo los cambios de dirección. Se estudió Theta* para producir diagonales, pero se descartó como modo predeterminado: incluso con muestras cada 0,5, un mapa táctil discreto no certifica todo el volumen continuo barrido por un robot 2x2. La compresión cardinal conserva la seguridad y obtuvo cero colisiones.

Control de movimiento

Cada tramo se ejecuta en lazo cerrado: se calcula el rumbo, se aplica el giro mínimo, se consulta contacto antes de cada paso y se detiene al alcanzar exactamente el objetivo. Todos los waypoints están alineados con el paso de 0,5 celdas.

Resultado representativo

Mapa	Origen - destino	Celdas A*	Tramos	Longitud	Error	Col.
cfg_0	(23.0, 7.0) - (2.0, 26.0)	81	3	40.0	0.0	0
cfg_1	(2.0, 2.0) - (23.0, 23.0)	85	4	42.0	0.0	0
cfg_2	(2.0, 2.0) - (39.0, 40.0)	151	4	75.0	0.0	0
cfg_3	(2.0, 2.0) - (12.0, 12.0)	41	4	20.0	0.0	0

4. Apartado C: teleoperación y rutas

Grabación

El usuario controla el robot con las flechas, añade waypoints con W y guarda con S o Q. Se almacena una ruta JSON versionada con nombre, escenario, fecha y coordenadas. Se omiten puntos consecutivos duplicados y la escritura usa un fichero temporal para evitar archivos parciales.

Reproducción autónoma

La reproducción no copia órdenes de teclado: carga cada waypoint y llama al planificador del apartado B entre la posición actual y el siguiente objetivo. Así puede rodear obstáculos, valida que mapa y ruta pertenezcan al mismo escenario y reutiliza exactamente el controlador ya probado.

Controles

Entrada	Acción
Arriba	Avanzar mientras está pulsada
Izquierda / derecha	Girar 15 grados
W	Añadir waypoint
S	Guardar ruta
Q	Añadir punto final, guardar y salir

Prueba de reproducción

La ruta de demostración contiene 5 waypoints y 4 desplazamientos principales. Finalizó con error 0.0, 0 colisiones y 100 frames de simulación.

Los ficheros inválidos, vacíos o pertenecientes a otro escenario se rechazan con un error explicativo.

5. Metodología y resultados

Se añadió una prueba unitaria para estado por instancia, A*, compresión del camino y persistencia; las pruebas de integración ejecutan cobertura, navegación y replay. El diagnóstico completo calcula el conjunto realmente alcanzable a partir de la geometría, pero ese oráculo está confinado a la evaluación.

Mapa	Poses	Celdas	Cobertura	Omitidas	Pasos	Colisiones
cfg_0	1308	416	100.0%	0	2614	0
cfg_1	1491	450	100.0%	0	2980	0
cfg_2	4417	1306	100.0%	0	8832	0
cfg_3	273	117	100.0%	0	544	0

Interpretación

Los cuatro escenarios alcanzan el 100% tanto en poses de medio paso como en superficie, sin omisiones ni visitas imposibles. El apartado A consulta el volumen frontal antes de moverse y termina con cero colisiones; B y C también terminan sin contacto.

Casos diagnosticados durante el desarrollo

- Posición inicial fuera de cfg_3 y mapa exportado de otro escenario.
- Estado mutable compartido entre instancias del simulador.
- Doble redondeo que generaba aristas no reversibles.
- Confusión entre aproximación bloqueada y celda globalmente ocupada.
- Atajos diagonales que no respetaban el volumen barrido del robot.

Criterios de aceptación: cobertura de poses=100%, cobertura de superficie=100%, omitidas=0, visitas imposibles=0, error final=0 y colisiones=0 en A/B/C.

6. Limitaciones y mejoras

Limitaciones

- La odometría del simulador es perfecta; no hay ruido, deriva ni localización probabilista.
- Los sensores son de contacto. El interior de obstáculos y regiones aisladas permanece honestamente desconocido.
- La cobertura DFS es completa pero puede recorrer cada arista del árbol dos veces y no minimiza energía.
- El mundo es estático; no se mantiene un mapa temporal de personas u objetos móviles.
- El sensor sólo informa del siguiente volumen frontal; no ofrece distancia ni geometría del obstáculo.

Mejoras con más tiempo

- Sustituir sondeos táctiles por lidar/sonar simulado y un mapa probabilista log-odds.
- Aplicar descomposición boustrophedon o Spanning Tree Coverage optimizado para reducir distancia.
- Añadir SLAM, estimación de pose y replanteo incremental D* Lite para obstáculos dinámicos.
- Optimizar la secuencia de fronteras por ganancia de información y coste de viaje.
- Integrar consumo de batería y retorno automático al cargador.

La elección final no persigue usar el algoritmo más complejo, sino el más avanzado que puede justificarse con la información sensorial disponible y comprobarse con invariantes claros.

7. Conclusiones

La práctica muestra que navegación, mapeo y control no deben resolverse de forma independiente. Un mapa visualmente plausible puede ser inútil para planificar si ignora la huella del robot; un camino geoméricamente corto puede ser inseguro si no se conoce su volumen barrido; y un seguidor de pared puede moverse indefinidamente sin cubrir el interior.

La solución final construye una abstracción común de espacio de configuración, garantiza terminación de la cobertura, usa búsqueda heurística óptima en la rejilla conocida y reutiliza el planificador durante la reproducción. La evaluación automatizada convierte afirmaciones como 'funciona' en medidas reproducibles.

Ejecución para la defensa

1. Ejecutar python validar_practica.py para mostrar las métricas. 2. Lanzar el apartado A con cfg_0 y enseñar el overlay. 3. Ejecutar una ruta larga del apartado B. 4. Grabar dos o tres waypoints y reproducirlos con el apartado C. 5. Explicar por qué se conserva UNKNOWN y por qué no se usan diagonales no certificadas.

Referencias

- [1] P. Hart, N. Nilsson y B. Raphael. A Formal Basis for the Heuristic Determination of Minimum Cost Paths. IEEE TSSC, 4(2), 1968. DOI: 10.1109/TSSC.1968.300136.
- [2] B. Yamauchi. A Frontier-Based Approach for Autonomous Exploration. IEEE CIRA, 1997. DOI: 10.1109/CIRA.1997.613851.
- [3] H. Choset. Coverage for Robotics - A Survey of Recent Results. Annals of Mathematics and Artificial Intelligence 31, 2001. DOI: 10.1023/A:1016639210559.
- [4] A. Nash, K. Daniel, S. Koenig y A. Felner. Theta*: Any-Angle Path Planning on Grids. AAAI, 2007, pp. 1177-1183.
- [5] Enunciado oficial Práctica P1: Navegación en entornos complejos, USC, curso 2026/2027.